Guía de laboratorios · IoT con Arduino para proyectos escolares

Oct 30, 2026 · @Oscar Enrique Llerena Castro

Cómo usar esta guía

La guía lleva al docente paso a paso por los 7 laboratorios del curso. La teoría está en los **Fundamentos teóricos**.

- Todos los enlaces del curso están en la página **ollerenac-uni.github.io/iot**: proyectos de Wokwi, prompts, tarjetas de prototipos y Forms.
- Cada laboratorio tiene cinco partes: **Objetivo**, **Qué se usa**, **Pasos**, **Prueba** y **Si algo falla**.
- Los pasos se hacen en orden. Cada paso es una acción.
- Los botones aparecen en **negrita** y con su nombre en inglés, tal como los muestra la pantalla.
- Las flechas (→) indican clics seguidos: **Channels → My Channels** es un clic en Channels y luego uno en My Channels.

El código del docente

- Cada docente tiene un código: **CL**, el número de su clase, un guion y los últimos 4 dígitos de su DNI.
- Ejemplo: clase 2, DNI terminado en 4821 → **CL2-4821**.
- El código nombra cada proyecto de Wokwi y cada canal de ThingSpeak. Nunca se usa el nombre de la persona.
- En esta guía, **CL#-####** significa «el código del docente».

Qué se entrega

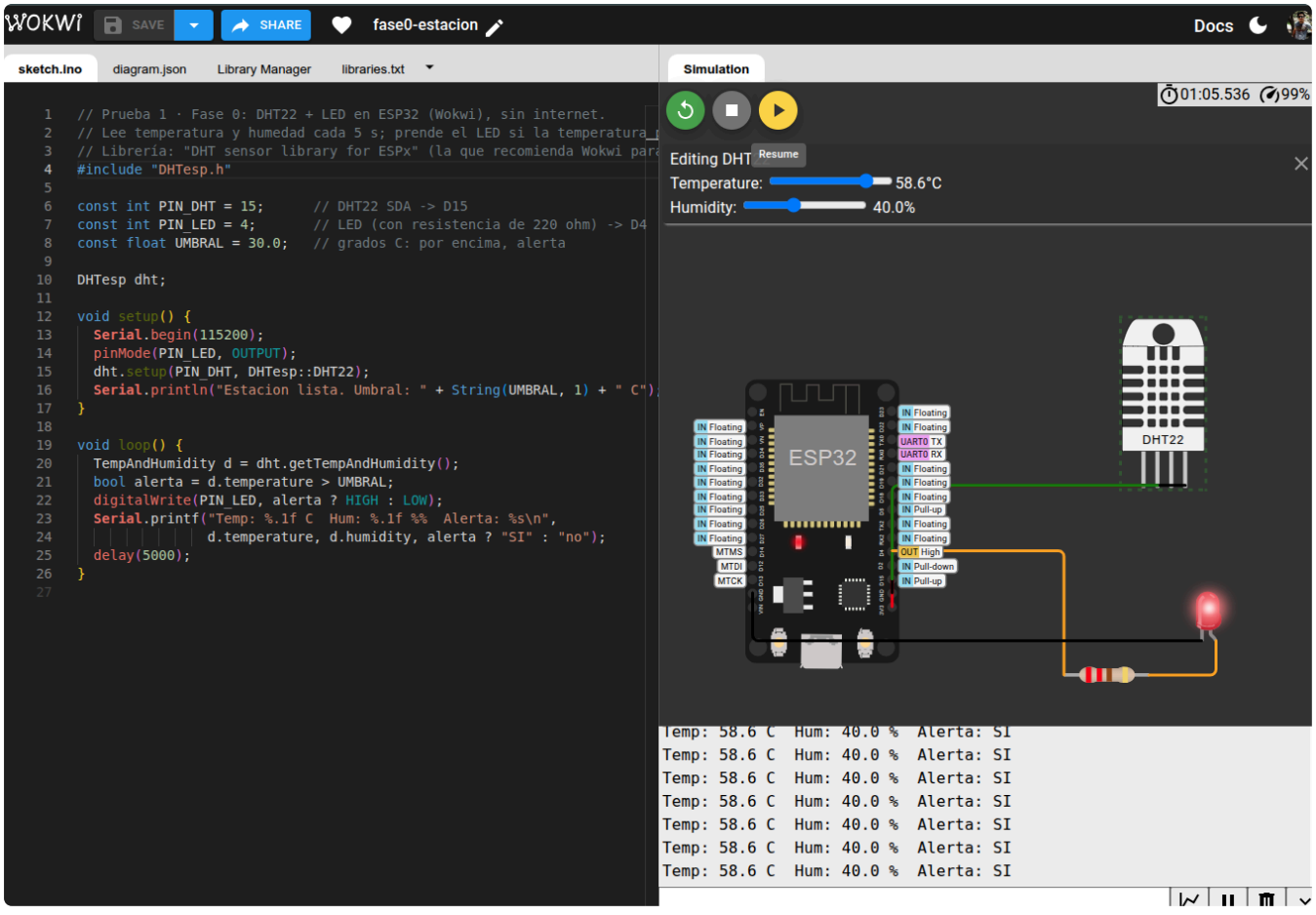
Laboratorio	Bloque	¿Se califica?
1 · Hola IoT	1	No: es práctica
2 · Semáforo de confort	2	No
3 · Proyecto con fallas	2	No
4 · Estación ambiental	3	Sí: prototipo 1
5 · Control de temperatura	3	Sí: prototipo 2

Laboratorio	Bloque	¿Se califica?
6 · Iluminación inteligente	4	Sí: prototipo 3
7 · Sistema de alerta	4	Sí: prototipo 4

- Cada prototipo calificado necesita su proyecto de Wokwi y su canal **público** de ThingSpeak.
- La cuenta gratis de ThingSpeak permite **4 canales**: uno por prototipo. El docente no crea canales de prueba.

Botones de Wokwi y ThingSpeak

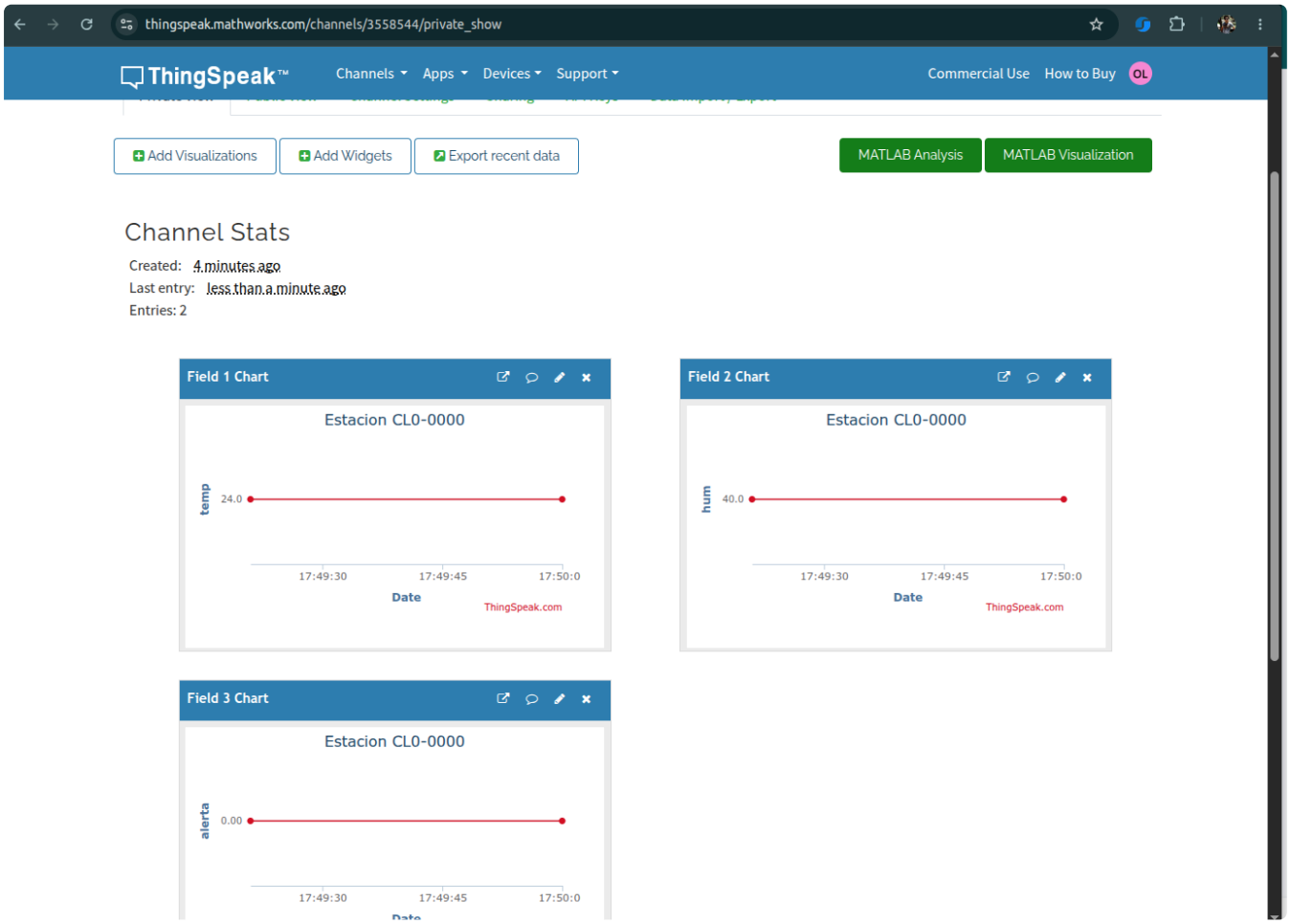
Los menús de las dos herramientas están en inglés. Esta tabla da el significado de cada botón que usa el curso.



Wokwi (wokwi.com, el simulador)

Botón o pestaña	Qué hace
Sign in	Entra con la cuenta de Gmail

Botón o pestaña	Qué hace
sketch.ino	Pestaña del código
diagram.json	Pestaña del circuito escrito como texto
Library Manager	Agrega librerías con el botón azul **+**
▶ (Start the simulation)	Compila y arranca la simulación
■ (Stop the simulation)	Detiene la simulación
+ azul del circuito	Agrega una pieza (LED, resistencia, sensor)
SAVE	Guarda el proyecto
▼ junto a SAVE → **Save a copy**	Copia un proyecto ajeno a la cuenta propia
SHARE	Da el enlace público del proyecto: es el que se entrega
Lápiz ✏️ junto al nombre	Cambia el nombre del proyecto
Clic en un sensor durante la simulación	Abre sus deslizadores (temperatura, luz, distancia)



ThingSpeak (thingspeak.mathworks.com, la nube)

Botón o pestaña	Qué hace
Sign In	Entra con correo y contraseña (no acepta Google)
Channels → My Channels	Lista de canales propios
New Channel	Crea un canal
Field 1, Field 2…	Campos del canal: cada uno guarda una variable
Save Channel	Guarda el canal
Private View	Gráficas del canal, vista del dueño
Public View	Gráficas que ve cualquiera, si el canal es público
Channel Settings	Cambia nombre y campos
Sharing → Share channel view with everyone	Hace público el canal
API Keys → Write API Key	Clave para escribir datos en el canal

Lista de revisión de lo que entrega la IA

El docente revisa la respuesta de la IA **antes** de cablear y de pegar. Se usa desde el laboratorio 2.

- [ ] Cada pin existe en el ESP32 y no se repite.
- [ ] Los sensores analógicos (luz, gas) van en **D32, D33, D34 o D35**.
- [ ] Cada pieza tiene alimentación (**3V3** o **VIN**) y **GND**.
- [ ] Cada LED tiene su resistencia de 220 Ω.
- [ ] Los pines del sketch son los mismos de la tabla.
- [ ] Las librerías existen y están agregadas en **Library Manager**.
- [ ] Los valores enviados a ThingSpeak no usan `String(valor, 0)`: llevan un espacio y ThingSpeak responde HTTP 400.
- [ ] La prueba se hace con un valor que **activa** y uno que **no activa**. Compilar no significa funcionar.

Si la IA se equivoca

- Error rojo al compilar: copiar el mensaje y pegarlo en el mismo chat: «Me sale este error, corrígelo sin cambiar los pines».
- Hace lo contrario de lo esperado: describir qué se esperaba y qué pasa, y pedir solo las líneas a cambiar.
- Caso real del curso: Gemini supuso que la oscuridad da valores bajos en la fotorresistencia. En Wokwi es al revés: 3938 con 1 lux y 156 con 10 000 lux (medido en Wokwi). El código compiló y la luz se habría prendido de día.

Laboratorio 1 · Hola IoT

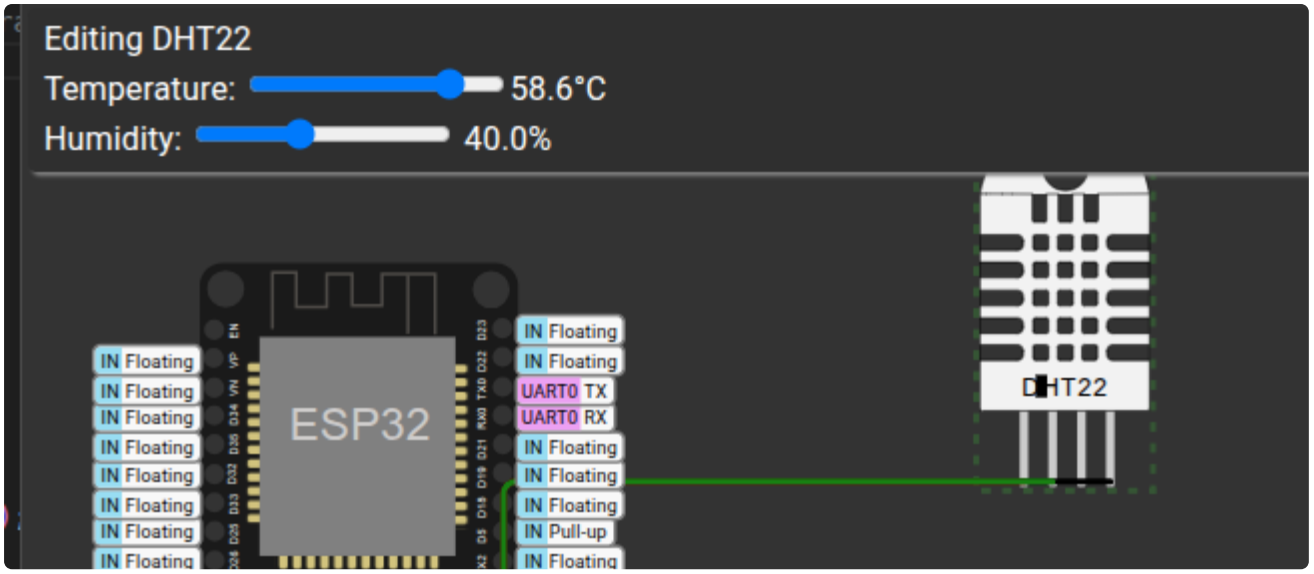
Objetivo: conectar un LED a mano, prenderlo cuando hace calor y enviar la temperatura a ThingSpeak.

Qué se usa

- Proyecto **Hola IoT** (página de enlaces → Bloque 1). Trae el ESP32 y el DHT22 conectados, y el código completo.
- Piezas que agrega el docente: un LED y una resistencia de 220 Ω.
- Cuenta de ThingSpeak.

Pasos: el LED

1. Abrir **Hola IoT** desde la página de enlaces.
2. Clic en ▼ junto a **SAVE** → **Save a copy**.
3. Cambiar el nombre con el lápiz ✏️: **Hola IoT CL#-####**.
4. Clic en el **+** azul del circuito → buscar **LED** → clic. Repetir con **Resistor**.
5. Arrastrar las dos piezas cerca de la placa.
6. Cablear con un clic en cada extremo:
   a. Pin **D4** del ESP32 → una pata de la resistencia.
   b. Otra pata de la resistencia → pata **A** del LED (la larga).
   c. Pata **C** del LED → **GND** del ESP32.
7. Pasar el mouse sobre un pin muestra su nombre.
8. Clic en ▶. La primera compilación espera en la cola: hasta 4 minutos.
9. Clic en el DHT22 → subir la temperatura con el deslizador por encima de 30 °C.



Pasos: la nube

1. En ThingSpeak: **Channels → My Channels → New Channel**.
2. **Name:** Estación CL#-####. **Field 1:** temperatura. **Field 2:** humedad.
3. Clic en **Save Channel**.
4. Pestaña **API Keys** → copiar la **Write API Key**.
5. En el sketch, pegar la clave en lugar de `PEGA_AQUI_TU_WRITE_API_KEY`, sin borrar las comillas.
6. Clic en **SAVE** y luego en ▶.
7. Pestaña **Sharing** → **Share channel view with everyone**.
8. Abrir **Public View** en el celular.
9. Cambiar `UMBRAL` en el código por otro valor → **SAVE** → ▶ → comprobar el nuevo valor con el deslizador.

Este canal «Estación» se reutiliza en el laboratorio 4.

Prueba

Acción	Resultado esperado
Temperatura sobre 30 °C	LED prendido; monitor serie: `LED prendido`
Temperatura bajo 30 °C	LED apagado
Cada 20 s	Monitor serie: `ThingSpeak: HTTP 200, respuesta 1`, `2`, `3`…
Private View	Las gráficas de temperatura y humedad suman un punto cada 20 s

Si algo falla

- **LED nunca prende:** revisar el cable a D4 y que la pata A vaya hacia la resistencia.
- **Error rojo:** se tocó el código sin querer. Abrir Hola IoT de nuevo y repetir **Save a copy**.
- **HTTP 400:** la clave está mal pegada.
- **HTTP 200, respuesta 0:** se envió antes de 15 s desde el dato anterior. El siguiente envío sí se guarda.
- **El WiFi no dice OK en 1 minuto:** detener con ■ y pulsar ▶ otra vez.

Laboratorio 2 · Semáforo de confort

Objetivo: usar la IA para diseñar y programar un semáforo que muestra el confort térmico del aula con un LED RGB.

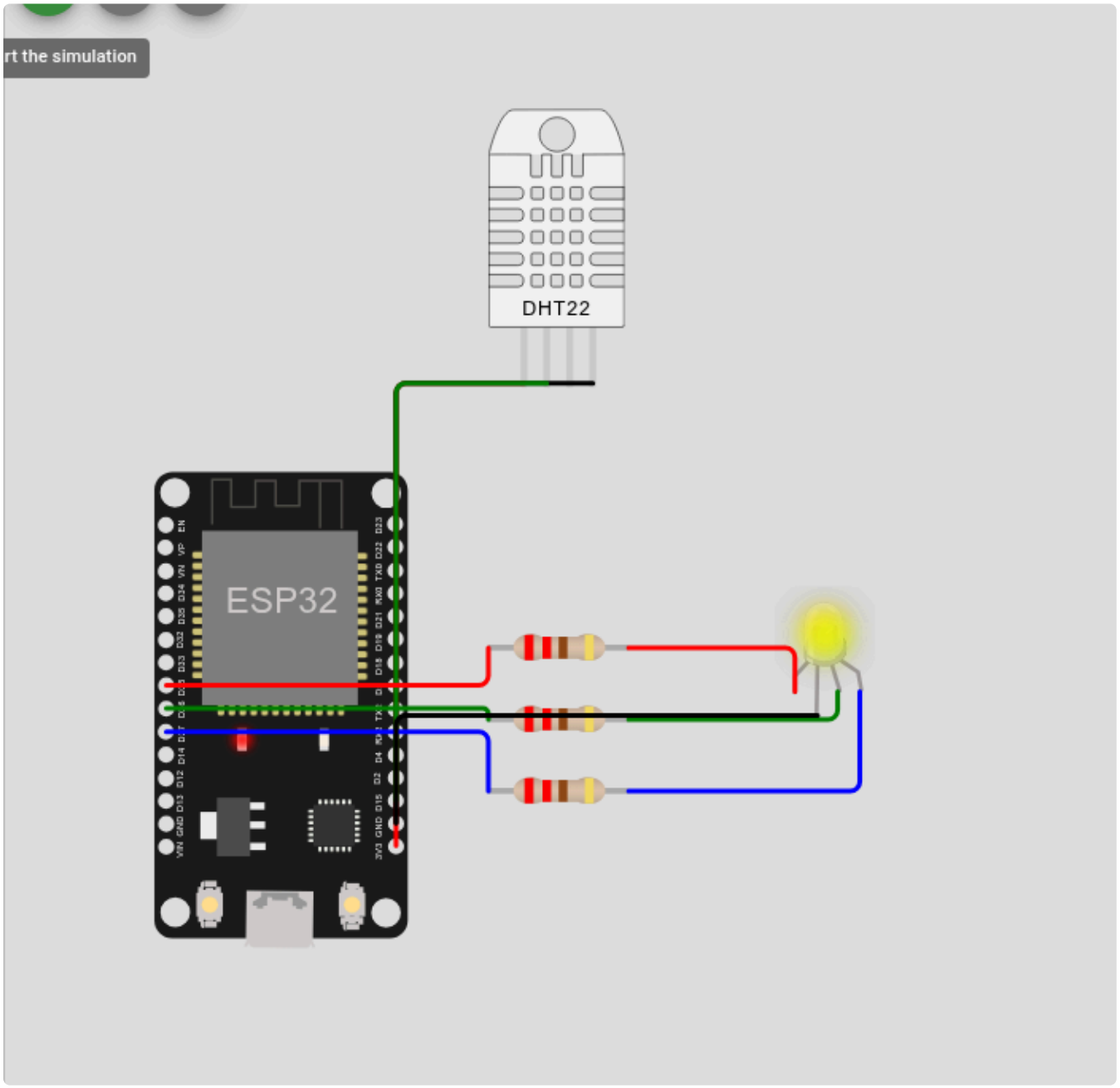
Qué se usa

- Proyecto **Semáforo de confort** (página de enlaces → Bloque 2). Trae el DHT22 conectado en D15 y un LED RGB de cátodo común, sin cables.
- Prompt del **Laboratorio 2** (página de enlaces → Prompts → botón Copiar).
- Gemini (gemini.google.com).
- Sin WiFi: este laboratorio no envía datos.

Pasos

1. Abrir el proyecto → ▼ junto a **SAVE** → **Save a copy** → nombre **Semáforo CL#-####**.
2. Copiar el prompt del Laboratorio 2 y pegarlo en un chat **nuevo** de Gemini.
3. Revisar la tabla de conexiones con la **lista de revisión**. La pata **COM** del RGB va a **GND**.

4. Agregar 3 resistencias con el **+** azul. El LED RGB ya está en el circuito: no se agrega otro.
5. Cablear según la tabla: cada color (R, G, B) pasa por su resistencia.
6. Pestaña **sketch.ino** → **Ctrl+A** → pegar el sketch de Gemini.
7. Clic en **SAVE** y luego en ▶.
8. Probar los tres rangos con el deslizador del DHT22.
9. Anotar cada corrección: qué falló, qué se cambió y por qué.



Prueba

Temperatura	Color esperado
20 °C	Verde
26 °C	Amarillo (rojo + verde)
30 °C	Rojo
Justo 24 °C y justo 28 °C	Depende de si el código usa `<` o `<=`: el docente decide cuál quiere

Si algo falla

- **Colores al revés o LED siempre prendido:** COM está en 3V3 en vez de GND.
- **Un color nunca prende:** falta su resistencia o su pin no coincide entre la tabla y el sketch.
- **El RGB se agregó con +:** Wokwi lo crea de ánodo común. En **diagram.json**, cambiar su atributo a `"common": "cathode"`.
- **Error rojo:** copiarlo a Gemini en el mismo chat.

Laboratorio 3 · Proyecto con fallas

Objetivo: encontrar y corregir tres fallas (una de código, una de conexión y una de lógica) con el simulador y la ayuda de la IA.

Qué se usa

- Proyecto **Proyecto con fallas** (página de enlaces → Bloque 2): DHT22, LED y resistencia.
- Prompt del **Laboratorio 3** (página → Prompts): sirve para depurar.
- Gemini.

Qué debería hacer el proyecto

- El DHT22 mide la temperatura cada 2 s.
- Sobre 30 °C se prende el LED rojo; si no, el LED queda apagado.
- El monitor serie muestra la temperatura y el estado del LED.

Pasos

1. Abrir el proyecto → **Save a copy** → nombre **Fallas CL#-####**.

2. Clic en ▶ y leer lo que pasa.
3. Ubicar el tipo de falla con las tres pistas (tabla de abajo).
4. Corregir una falla → **SAVE** → ▶ → probar otra vez.
5. Si no se encuentra la causa, usar el prompt del Laboratorio 3: pegar el sketch, describir los cables, decir qué se esperaba y qué se ve.
6. Pedir a la IA **solo las líneas a cambiar**, con el antes y el después.
7. Repetir hasta que el proyecto haga lo que debería.
8. Anotar cada falla: dónde estaba, cómo se descubrió, cómo se corrigió.

Tres pistas para ubicar una falla

Lo que se ve	Tipo de falla	Dónde buscar
Error rojo al compilar	Código	La línea que indica el mensaje
Compila, pero una pieza no responde	Conexión	Pin, alimentación y GND de esa pieza; comparar con los pines del código
Responde, pero al revés o a destiempo	Lógica	El umbral y la comparación (`<` , `>`)

Prueba

Temperatura	Resultado esperado al final
35 °C	LED prendido
20 °C	LED apagado

Si algo falla

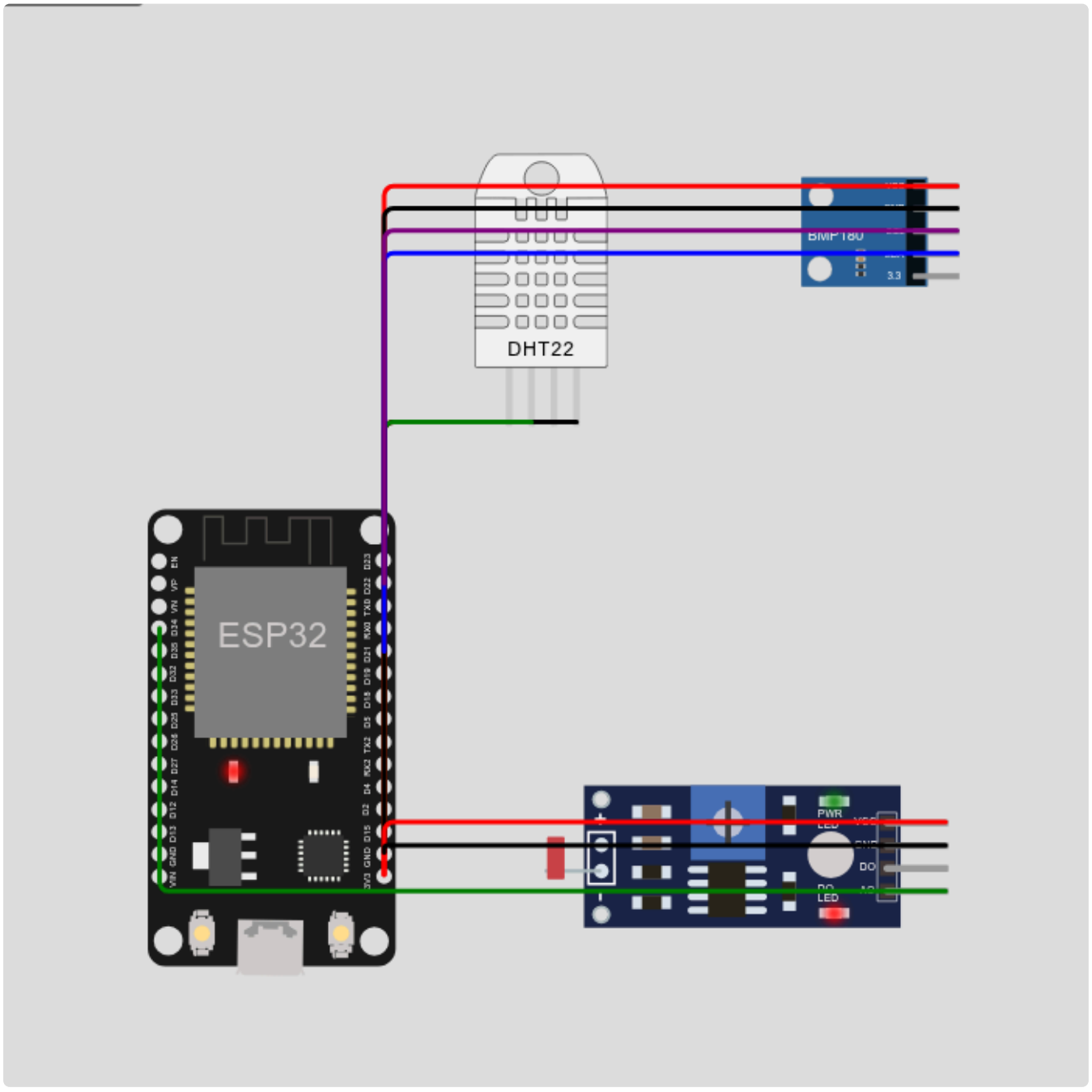
- **La IA reescribe todo el programa:** pedirle de nuevo solo las líneas a cambiar.
- **Se arregló una falla y aparece otra:** es lo esperado. Las tres fallas aparecen una detrás de otra.

Laboratorio 4 · Estación ambiental (prototipo 1)

Objetivo: medir temperatura, humedad, presión y luz del aula y enviarlas a ThingSpeak. Se califica.

Qué se usa

- Página de enlaces → **Prototipos** → Estación ambiental → **Proyecto de partida**: DHT22, BMP180 y fotorresistencia (LDR), sin cables.
- Prompt del **Laboratorio 4**.
- Canal **Estación CL#-####** del laboratorio 1.



Conexiones de referencia

Pieza	Conexión	Señal
DHT22	VCC → 3V3 · GND → GND · SDA → D15	Digital

Pieza	Conexión	Señal
BMP180	VCC → 3V3 · GND → GND · SDA → **D21** · SCL → **D22**	I2C
LDR	VCC → 3V3 · GND → GND · AO → **D34**	Analógica

Pasos

1. Abrir el proyecto de partida → **Save a copy** → nombre **Estación CL#-####**.
2. Copiar el prompt del Laboratorio 4 en un chat nuevo de Gemini.
3. Revisar la tabla de la IA contra la tabla de referencia. La LDR va solo en D32 a D35.
4. Cablear los 10 cables: alimentación, tierra y datos de los tres sensores.
5. **Library Manager** → **+** → agregar **DHT sensor library for ESPx** y **Adafruit BMP085 Library**.
6. En ThingSpeak, canal Estación → **Channel Settings** → marcar **Field 3** (presión) y **Field 4** (luz) → **Save Channel**.
7. Pegar el sketch de la IA y la **Write API Key** del canal Estación.
8. **SAVE** → ▶.
9. Clic en cada sensor y mover sus deslizadores.
10. Dejar la simulación corriendo al menos **4 minutos** (10 datos o más por campo).
11. **SHARE** → copiar el enlace del proyecto.

Orden de los campos (así se califica)

Campo	Variable	Unidad
Field 1	Temperatura	°C
Field 2	Humedad	%
Field 3	Presión	hPa
Field 4	Luz	%

Prueba

- Monitor serie, una línea cada 20 s, por ejemplo: `Temp 24.0 C | Hum 40.0 % | Presion 1013.3 hPa | Luz 75 %`.
- Mover un deslizador cambia su valor en la línea siguiente.
- **Private View**: las 4 gráficas suman un punto cada 20 s.
- Menos luz en la LDR → menor porcentaje de luz.

Si algo falla

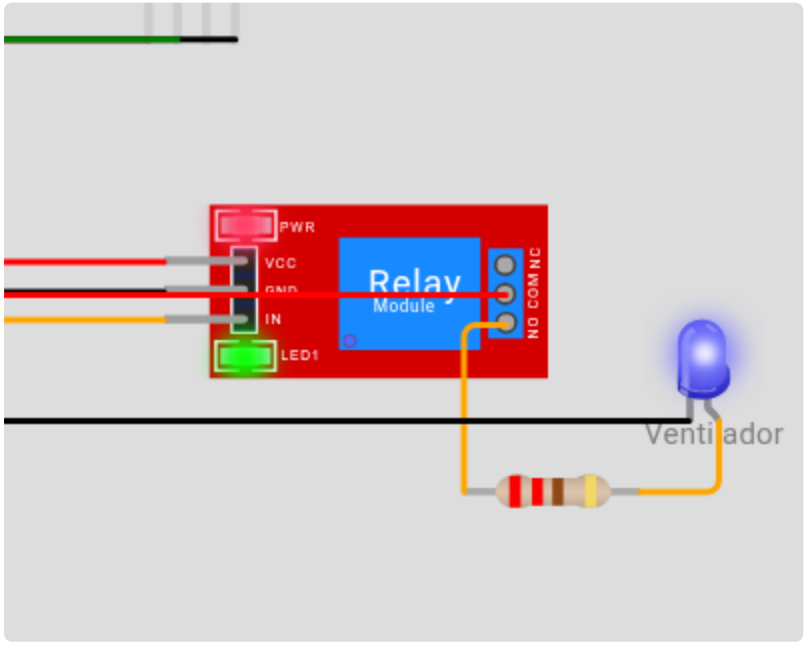
- **«No encuentro el BMP180»:** SDA y SCL cruzados o sin 3V3.
- **Luz siempre 0 o 100:** la LDR no está en un pin de D32 a D35 o no tiene alimentación.
- **Luz al revés (sube con la oscuridad):** el código debe usar `map(analogRead(pin), 4095, 0, 0, 100)`.
- **HTTP 400:** clave mal pegada o un valor con espacio.
- **Se atrasó el docente:** usar el **Proyecto de rescate**. Está cableado y envía temperatura y humedad; falta leer la presión y la luz (líneas marcadas `FALTA`).

Laboratorio 5 · Control de temperatura (prototipo 2)

Objetivo: prender un ventilador con un relé cuando hace calor y apagarlo cuando refresca, con dos umbrales (histéresis). Se califica.

Qué se usa

- Página → **Prototipos** → Control de temperatura → **Proyecto de partida**: DHT22, módulo relé, resistencia y LED azul «Ventilador», sin cables.
- Prompt del **Laboratorio 5**.
- Un canal nuevo: **Control CL#-####**.



Conexiones de referencia

Pieza	Conexión	Lado
DHT22	VCC → 3V3 · GND → GND · SDA → D15	—

Pieza	Conexión	Lado
Relé	VCC → **VIN** · GND → GND · IN → **D26**	Control
Carga	COM → 3V3 · NO → resistencia → A del LED · C del LED → GND	Carga

Los dos umbrales

Temperatura	Si estaba apagado	Si estaba prendido
Sobre el umbral alto	Se prende	Sigue prendido
Entre los dos umbrales	Sigue apagado	Sigue prendido
Bajo el umbral bajo	Sigue apagado	Se apaga

- El modelo del facilitador usa **28 °C** (alto) y **26 °C** (bajo).
- Cada docente elige **sus propios** umbrales, distintos de 28 y 26. Es un criterio de calificación.

Pasos

1. Abrir el proyecto de partida → **Save a copy** → nombre **Control CL#-####**.
2. Escribir los dos umbrales propios en el prompt del Laboratorio 5 y enviarlo a Gemini.
3. Revisar la tabla y cablear el relé y su carga.
4. **Library Manager** → **+** → **DHT sensor library for ESPx**.
5. ThingSpeak → **New Channel** → **Name:** Control CL#-#### · **Field 1:** temperatura · **Field 2:** relé → **Save Channel**.
6. Pegar el sketch y la **Write API Key** del canal Control → **SAVE** → ▶.
7. Hacer la prueba de abajo **sin detener la simulación**, esperando 20 s entre cada cambio.
8. **Sharing** → **Share channel view with everyone**.
9. **SHARE** → copiar el enlace del proyecto.

Prueba (en una misma corrida)

Acción	Resultado esperado
Subir sobre el umbral alto	Relé y LED Ventilador prendidos; campo 2 = 1
Bajar a un valor entre los dos	Siguen prendidos

Acción	Resultado esperado
Bajar bajo el umbral bajo	Se apagan; campo 2 = 0

- En **Private View**, la gráfica del campo 2 debe subir a 1 y volver a 0.

Si algo falla

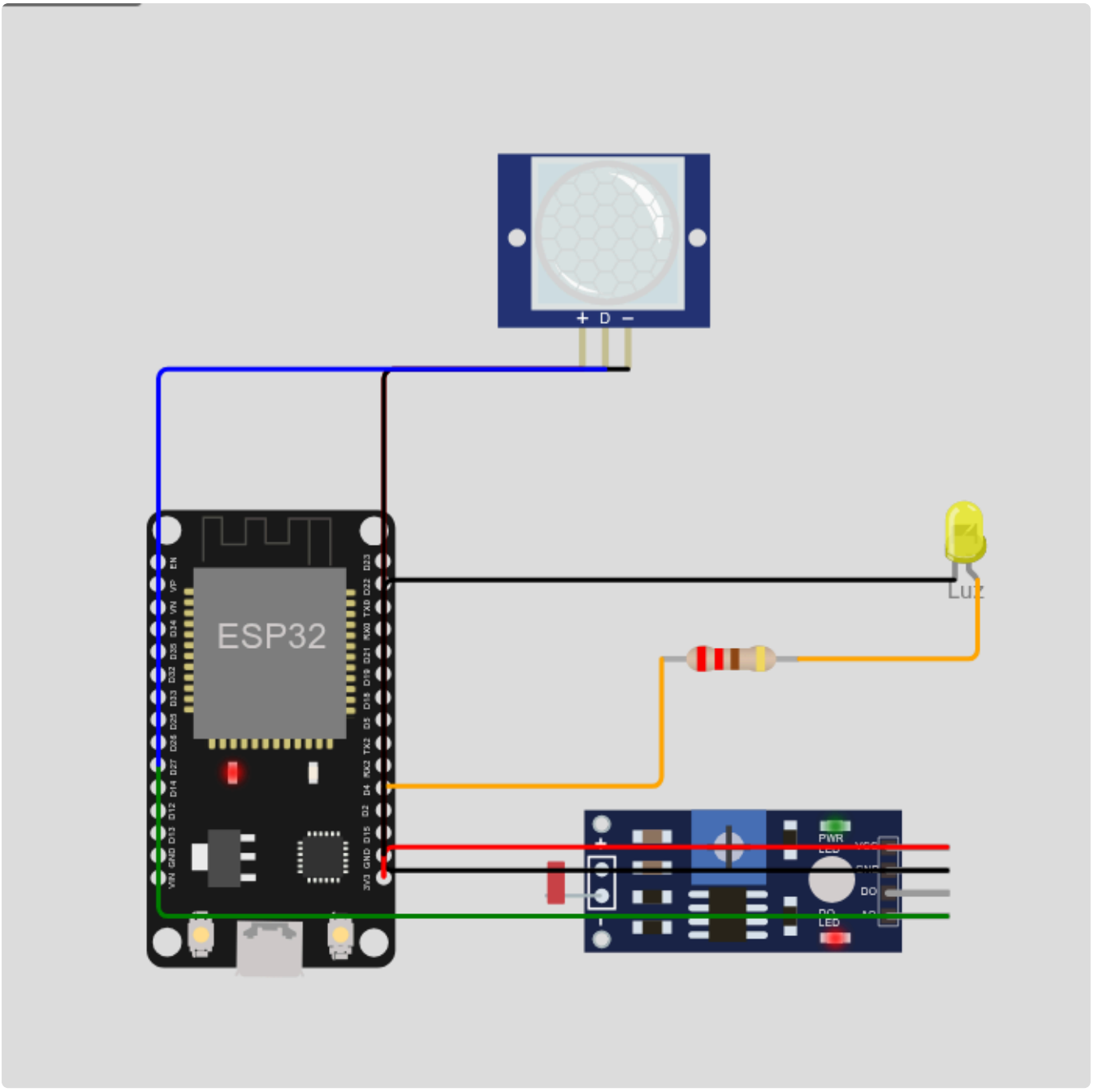
- **El LED verde del relé no prende:** revisar VCC en VIN e IN en D26.
- **El relé cambia pero el LED no prende:** revisar el lado de carga (COM a 3V3, NO a la resistencia).
- **Prende y apaga sin parar cerca de un valor:** el código usa un solo umbral. Pedir a la IA la histéresis con dos.
- **Se atrasó el docente: Proyecto de rescate**. Está cableado y envía; faltan los umbrales y la decisión (líneas `FALTA`).

Laboratorio 6 · Iluminación inteligente (prototipo 3)

Objetivo: prender la luz de un pasillo solo si está oscuro **y** hay alguien. Desde este laboratorio, la IA también entrega el circuito (diagram.json). Se califica.

Qué se usa

- Página → **Prototipos** → Iluminación inteligente → **Proyecto de partida**: solo la placa ESP32.
- Prompt del **Laboratorio 6**: pide tabla, diagram.json y sketch.
- Un canal nuevo: **Iluminación CL#-####**.



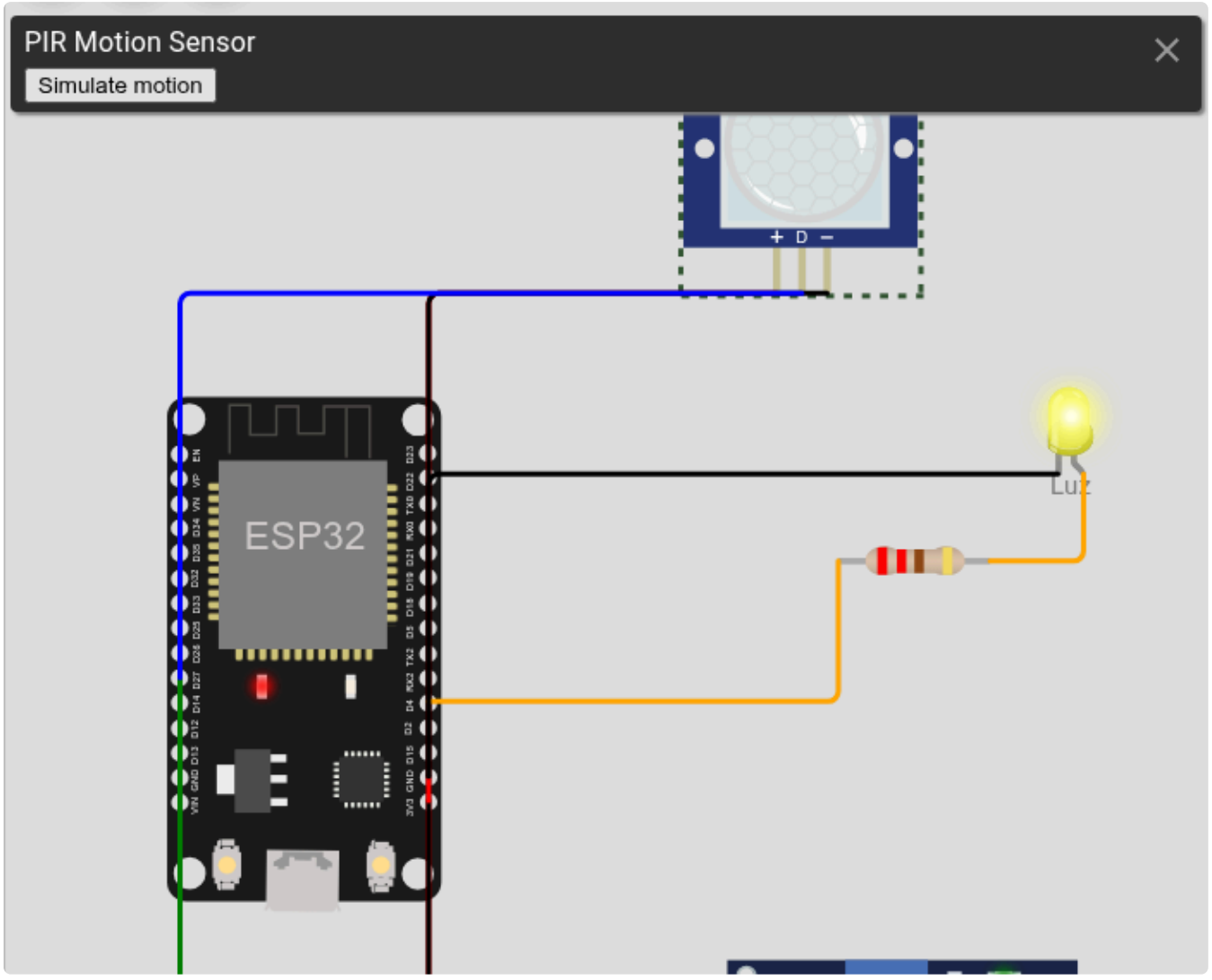
Conexiones de referencia

Pieza	Conexión	Señal
LDR	VCC → 3V3 · GND → GND · AO → D34	Analógica
PIR	VCC → 3V3 · GND → GND · OUT → D27	Digital
LED	D4 → resistencia 220 Ω → A · C → GND	Digital

Pasos

1. Abrir el proyecto de partida → **Save a copy** → nombre **Iluminación CL#-####**.

2. Escribir el umbral de oscuridad propio en el prompt (el modelo usa 30 % de luz; elegir otro valor) y enviarlo a Gemini.
3. Pestaña **diagram.json** → **Ctrl+A** → pegar el diagram.json de la IA. El circuito aparece dibujado.
4. Revisar el circuito con la **lista de revisión**. Corregir a mano o pedir la corrección a la IA.
5. Pestaña **sketch.ino** → **Ctrl+A** → pegar el sketch.
6. ThingSpeak → **New Channel** → **Name:** Iluminación CL#-#### · **Field 1:** luz · **Field 2:** movimiento · **Field 3:** encendida → **Save Channel**.
7. Pegar la **Write API Key** → **SAVE** → ▶.
8. Hacer la prueba de abajo. Para simular a una persona: clic en el PIR → **Simulate motion**.
9. **Sharing** → **Share channel view with everyone**.
10. **SHARE** → copiar el enlace del proyecto.



Prueba

Luz	Movimiento	Resultado esperado
Oscuro (deslizador de lux al mínimo)	Sí	**Luz prendida**; campo 3 = 1
Oscuro	No	Apagada
Claro	Sí	Apagada
Claro	No	Apagada

- Primero se oscurece la LDR y después se pulsa **Simulate motion**.
- Con la luz inicial (75 %), el movimiento solo **no** prende la luz: es lo correcto.
- El monitor serie escribe solo al enviar: cada 20 s o cuando la luz cambia.

Si algo falla

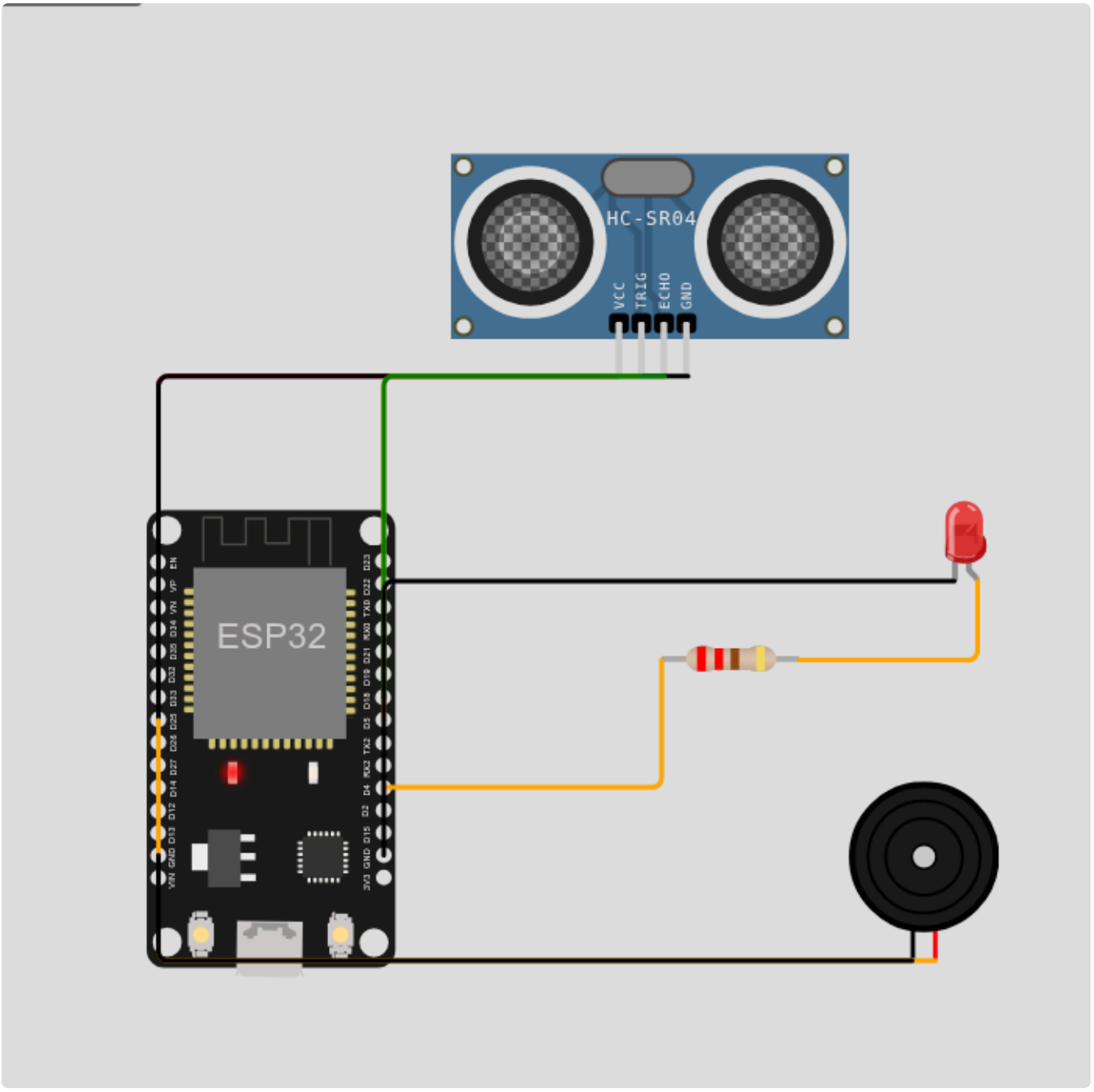
- **La luz se prende de día:** la lectura de la LDR está invertida. En Wokwi el valor sube con la oscuridad.
- **El circuito no aparece al pegar:** el diagram.json está incompleto. Pedirlo de nuevo a la IA, completo.
- **Se atrasó el docente: Proyecto de rescate**. Está cableado y envía; faltan el umbral y la decisión (líneas `FALTA`).

Laboratorio 7 · Sistema de alerta (prototipo 4)

Objetivo: dar una alerta local (zumbador y LED) y una alerta remota (ThingSpeak) cuando el tanque de agua del colegio baja de un nivel mínimo. Se califica.

Qué se usa

- Página → **Prototipos** → Sistema de alerta → **Proyecto de partida**: solo la placa ESP32.
- Prompt del **Laboratorio 7**: pide tabla, diagram.json y sketch.
- Un canal nuevo: **Alerta CL#-####**.
- Opciones de sensor: ultrasónico HC-SR04 (tanque, el del prompt), gas MQ2 o movimiento PIR. Con otro sensor, el docente cambia el contexto del prompt.



Conexiones de referencia (tanque)

Pieza	Conexión	Señal
HC-SR04	VCC → **VIN** · GND → GND · TRIG → D5 · ECHO → D18	Digital
Zumbador	pata 2 (+) → D25 · pata 1 (−) → GND	Digital
LED rojo	D4 → resistencia 220 Ω → A · C → GND	Digital

Cómo se calcula el nivel

- El sensor está en la tapa del tanque: más distancia al agua = menos agua.

- Distancia (cm) = tiempo del eco (µs) ÷ 58.
- Nivel (%) = 100 × (1 − distancia ÷ altura del tanque).
- El modelo del facilitador usa un tanque de **100 cm** y alerta bajo **20 %**. Cada docente elige su propio nivel mínimo.

Pasos

1. Abrir el proyecto de partida → **Save a copy** → nombre **Alerta CL#-####**.
2. Escribir la altura del tanque y el nivel mínimo propios en el prompt y enviarlo a Gemini.
3. Pegar el diagram.json y revisarlo con la **lista de revisión**.
4. Pegar el sketch.
5. ThingSpeak → **New Channel** → **Name:** Alerta CL#-#### · **Field 1:** nivel · **Field 2:** alerta → **Save Channel**.
6. Pegar la **Write API Key** → **SAVE** → ▶.
7. Clic en el ultrasónico → mover el deslizador de distancia.
8. **Sharing** → **Share channel view with everyone**.
9. **SHARE** → copiar el enlace del proyecto.

Prueba (tanque de 100 cm, mínimo 20 %)

Distancia	Nivel	Resultado esperado
30 cm	70 %	Sin alerta; campo 2 = 0
90 cm	9 %	Suenan el zumbador y el LED; campo 2 = 1

- Valores medidos en Wokwi con el modelo del facilitador.
- En **Private View**, el campo alerta debe llegar a 1 al menos una vez.

Si algo falla

- **El nivel no cambia con el deslizador:** revisar TRIG en D5, ECHO en D18 y VCC en VIN.
- **HTTP 400 solo con niveles de una cifra:** el código usa `String(nivel, 0)`. Cambiarlo por `String(lround(nivel))`.
- **Sensor de gas MQ2:** en Wokwi solo responde a su deslizador; con aire limpio ya marca 3628 de 4095.
- **Se atrasó el docente: Proyecto de rescate**. Está cableado y envía el nivel; faltan el nivel mínimo, la decisión y la alarma local (líneas `FALTA`).

Entrega del producto

Plazo: hasta las 23:59 del día del curso, por el **Form de entrega** (página de enlaces → Bloque 4). Vale el último envío.

Antes de enviar

- [ ] Los 4 proyectos de Wokwi están guardados con **SAVE**.
- [ ] Cada proyecto tiene su enlace de **SHARE** copiado.
- [ ] Los 4 canales de ThingSpeak son públicos (**Sharing → Share channel view with everyone**).
- [ ] Cada nombre lleva el código **CL#-####** y ningún dato personal.
- [ ] Los umbrales son propios, distintos del modelo, como constantes con nombre y comentario.
- [ ] Cada enlace abre en una **ventana de incógnito**. Si no se ve sin la cuenta, el facilitador tampoco lo ve.

El Form pide

1. El código CL#-####.
2. Por cada prototipo: el enlace de Wokwi y el enlace del canal de ThingSpeak.
3. En el sistema de alerta: el sensor usado (nivel del tanque, gas o movimiento).
4. La declaración: el código y los diagramas de la IA se revisaron y probaron, y los canales no tienen datos personales.

Cómo se califica

Prototipo	Criterio del dispositivo (Wokwi)	Criterio de los datos (ThingSpeak)
Estación ambiental	Abre y simula con los tres sensores leyendo	4 campos con 10 datos o más
Control de temperatura	Relé cableado y dos umbrales	El relé pasa a 1 sobre el alto y a 0 bajo el bajo
Iluminación inteligente	LDR, PIR y LED; prende solo con oscuridad y movimiento	Luz encendida = 1 solo con movimiento y poca luz
Sistema de alerta	Sensor, zumbador y LED; alerta local	Alerta = 1 al menos una vez

- Dos criterios generales: los 4 canales públicos con el código y sin datos personales; y umbrales propios, distintos del modelo.
- 10 criterios de sí o no, 2 puntos cada uno. Aprueba quien cumple 6 de 10 (12 puntos) y marcó los 4 check-ins.

Al terminar el curso

- En cada canal: **API Keys** → generar una clave nueva. La clave vieja quedó visible en el proyecto público de Wokwi.